Annexe A19 — La performance de Render

LibraryV3 · Phase G, étape 4 · de la Référence K à la Référence L · 11 octobre 2026 · un modèle de coût, un plan d'expérience, une décision

Résultat en une ligne. `Render` passe de **≈ 595 µs** (Référence K) à **≈ 204 µs** (Référence L), soit **÷ 2,9**, et la frame de 2,24–2,48 ms à 2,01–2,05 ms. La cause n'était ni le rasterizer ni le frustum : **98,5 % des objets dessinés faisaient moins d'un pixel**, et chacun payait la transformation de ses sommets et le test de ses faces pour n'allumer, presque toujours, aucun pixel. Le remède, le dessin en point, fait l'objet de l'annexe A20.

1. Le point de départ : la Référence K

Après le chantier Clear (A15), la Référence K (i7-1265U épinglé `affinity=4`, Release, 4 vues, 1536 × 900, scène `solar_system_v1compat_belt_lod_spheres.json`) donnait, en médianes par frame :

Compteur	Valeur
Render	569 / 599 / 618 µs (3 runs)
Meshes testés / rejetés par le frustum	2 580 / 1 283
Meshes dessinés	≈ 1 297
Faces soumises	≈ 14 752 (≈ 11 par mesh dessiné)
Triangles tournés vers la caméra	≈ 7 190
… rejetés tôt (couverture nulle, chantier 3b)	≈ 6 890
Triangles qui touchent réellement des pixels	**≈ 300**

96 % des triangles tournés vers la caméra n'allumaient aucun pixel. La rasterisation proprement dite était devenue marginale : le temps partait **avant** le premier pixel. C'est ce qu'on a appelé le **coût fixe** : ce qu'on paie même quand un objet ne produit rien.

2. Un modèle avant toute optimisation

```
Render ≈ k + a·(meshes rejetés) + b·(meshes dessinés) + c·(faces soumises) + [pixels]
```

`k` est la constante par frame (vues, gizmos), `a` le coût d'un mesh testé puis rejeté par le frustum, `b` le coût fixe d'un mesh dessiné (matrice, sélection du niveau, préparation), `c` le coût d'une face (sa part de sommets transformés, orientation, test de couverture).

> × Calculer a, b et c par moindres carrés sur les 720 frames de la Référence K. Résultat absurde : une constante de −6,4 ms, 5 µs par mesh, des corrélations inférieures à 0,25. Dans ces runs, les compteurs ne bougent presque pas (meshes dessinés de 1 293 à 1 304, ±0,4 %) : on ne sépare pas des termes qui varient ensemble, et de si peu. **Une mesure passive ne sépare pas des causes : il faut un plan d'expérience, qui fait varier un seul facteur à la fois** (règle M4).

3. La campagne R : sept scènes synthétiques

Quatre caméras **identiques** (même pose, même objectif) : chaque vue voit les mêmes objets. Rien ne bouge (vitesses nulles, aucun contrôleur) : les compteurs sont constants d'une frame à l'autre, seul le temps varie. Les objets sont sur une grille à 200 unités, d'un diamètre de 0,1 unité (≈ 0,35 px) : **sous-pixel par construction**, pour mesurer le coût fixe sans pixels.

Scène	Contenu	Terme isolé
S0	4 caméras, aucun mesh	`k`
S1_300 / S1_1200	rochers (`rock_a.obj`) **derrière** les caméras	`a`
S2_300 / S2_1200	rochers visibles, 168 faces chacun	`b + 168 c`
S3_300 / S3_1200	cubes visibles, 12 faces chacun	`b + 12 c`

Les rochers sont chargés en `.obj`, pas en `.lod.json` : le nombre de faces est une variable **contrôlée**. Chaque scène est lancée deux fois, dans l'ordre aller puis retour (S0 → S3_1200 → … → S0), pour qu'une dérive thermique touche chaque scène de façon symétrique.

3.1 Les compteurs d'abord, les temps ensuite

Avant de lire un seul temps, on vérifie que l'expérience est bien celle qu'on a voulue : sur les 240 frames mesurées de chaque run, les compteurs sont **strictement constants** et conformes au plan (S1 : tous rejetés ; S2 : 4N × 168 faces ; S3 : 4N × 12 ; S0 : les 12 gizmos). En S2 et S3, 99,9 % des triangles tournés vers la caméra sont rejetés tôt : la condition « sous-pixel » est tenue. Les runs a et b diffèrent de moins de 4,2 % sur `Render`.

3.2 Les résultats

Scène	Render (moyenne a/b)
S0	7,2 µs
S1_300 / S1_1200	30,2 / 98,8 µs
S2_300 / S2_1200	3 104 / 12 165 µs
S3_300 / S3_1200	375 / 1 331 µs

Passer de N = 300 à N = 1 200 ajoute 900 objets par vue, soit 3 600 meshes. La pente donne le coût unitaire :

Terme	Calcul	Coût
`a` : mesh rejeté	(98,8 − 30,2) / 3 600	**19 ns**
rocher dessiné (168 faces)	(12 165 − 3 104) / 3 600	2 517 ns
cube dessiné (12 faces)	(1 331 − 375) / 3 600	266 ns
`c` : par face	(2 517 − 266) / 156	**14,4 ns**
`b` : par mesh dessiné, hors faces	266 − 12 × 14,4	**92 ns**

Contrôle de linéarité : prolongée jusqu'à N = 0, la droite de S1 donne 7,4 µs, soit exactement S0 (7,2 µs). Le modèle tient sans terme caché. On ne peut pas séparer le coût d'une face de celui d'un sommet, car les deux varient ensemble dans nos meshes : les 14,4 ns par face incluent leur part de transformation des sommets.



Figure 1 — Coût d'un objet qui n'allume AUCUN pixel (campagne R, pente entre N = 300 et N = 1 200). Le rejet par le frustum est déjà bon marché ; l'objet visible mais minuscule coûte 14 à 132 fois plus.

3.3 Une observation sur Clear

Dans les fichiers de synthèse, `Clear` **baisse quand la frame s'allonge** : 410 à 460 µs en S0 et S1 (frames ≈ 1,8 ms), 310 à 350 µs en S2 (4 à 14 ms), alors que le buffer à effacer a la même taille. L'explication la plus probable : `Present` envoie l'image précédente pendant que la frame suivante commence ; quand la frame est longue, l'envoi est terminé avant le `Clear` suivant, qui dispose seul de la bande passante. On ne compare donc les `Clear` qu'entre scènes de durée voisine (règle M3).

4. Appliquer le modèle à la vraie scène

Terme	Calcul	Coût
constante		7 µs
meshes rejetés	1 283 × 19 ns	24 µs
meshes dessinés	1 297 × 92 ns	120 µs
faces soumises	14 752 × 14,4 ns	213 µs
prédit		**364 µs**
mesuré		**≈ 600 µs**

Le modèle explique 61 % de `Render` : du travail payé **avant le moindre pixel**. Le reste vient des vrais pixels (Soleil, planètes proches), du découpage des objets à cheval sur le frustum, et des caches : dans R, un seul mesh était réutilisé 1 200 fois, ce qui est le cas le plus favorable.

5. Le comptage : qui sont ces objets ?

Un run de comptage (`LV3_LOD_STATS=1`, temps non recevables) a classé chaque paire (objet, vue) dessinée selon le **rayon** à l'écran de sa sphère englobante :

Rayon à l'écran	Objets dessinés	Faces soumises
< 1 px	**1 278 (98,5 %)**	**13 628 (92 %)**
1 à 4 px	14	628
4 à 16 px	3	192
≥ 16 px	2	304

`LodLevel3` valait 1 285 : le LOD faisait déjà tout ce qu'il pouvait, ces objets étaient tous à leur niveau le plus grossier. Leur coût, d'après le modèle : 1 278 × 92 ns + 13 628 × 14,4 ns ≈ **314 µs**, plus de la moitié de `Render`.

> × Le rejet par contribution de manuel : si `rPx < 1`, ne rien dessiner. On gagnerait les 314 µs, mais **la ceinture d'astéroïdes disparaîtrait** : ces 1 278 objets sont précisément ses rochers. Le rejet par contribution vise des objets dont l'absence ne se voit pas ; ici, l'objet sous-pixel est la scène elle-même. On ne supprime pas un objet sous-pixel : on change la façon de le dessiner (A20).

6. La Référence L

Lot de 3 runs, mêmes conditions que K, avec le dessin en point actif (`lod.pointRadiusPx = 1.0`) :

Contrôle	K	L	Verdict
`Σ LodLevel + PointSplats = testés − rejetés`	—	0 frame en défaut sur 720	✔
T = Animation + Xform + Trigger (M3)	95,7 à 102,4 µs	100,2 à 101,4 µs	✔
Clear	461 à 546 µs	445 à 462 µs	✔ comparable
PointSplats	—	1 277 à 1 282	✔ = le comptage

Médiane	K	L	Écart
Render	569 / 599 / 618 µs	**203 / 209 / 201 µs**	**−66 %**
Faces soumises	14 752	1 388	−91 %
Sommets transformés	20 453	1 352	−93 %
Frame	2,24 à 2,48 ms	2,01 à 2,05 ms	≈ −15 %

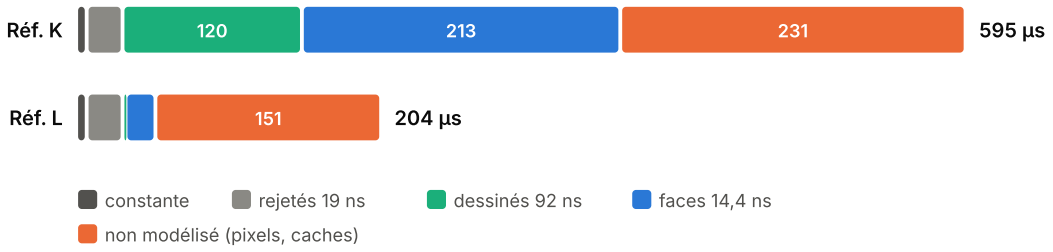


Figure 2 — Render décomposé par le modèle de la campagne R. Référence K : le modèle explique 61 %. Référence L : les 1 280 objets sous-pixel ne passent plus par les faces ; le reste (points, vrais pixels, caches) n'est plus séparable sans RASTER_STATS. Valeurs en µs, médianes.

Prévision et mesure. Le modèle prévoyait 314 µs de gain ; on mesure ≈ 390 µs, soit ≈ 305 ns par objet transformé en point, contre 245 ns dans les scènes synthétiques. L'écart est celui que le banc annonçait : dans R, un seul mesh restait chaud dans le cache L1 ; dans la vraie scène, trois rochers différents, leurs chaînes de LOD et des entités dispersées coûtaient environ 25 % de plus en défauts de cache. Le point supprime ces accès en même temps que les calculs.

> **À retenir.** Un banc synthétique donne un **minorant** des coûts ; la vraie scène tranche. On mesure deux fois : d'abord pour **comprendre** (un plan d'expérience, une variable à la fois), ensuite pour **valider** (la scène de référence, dans les mêmes conditions).

7. La nouvelle répartition de la frame

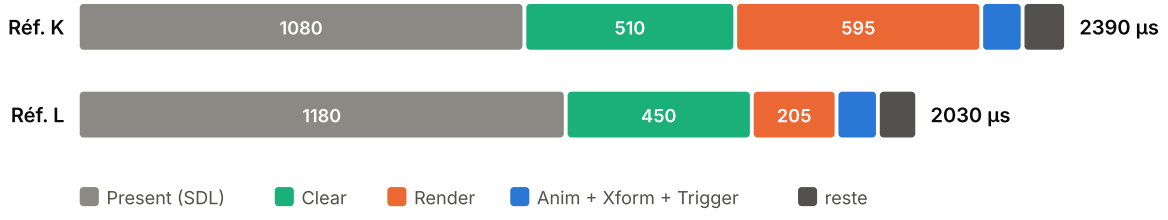


Figure 3 — Répartition de la frame (médianes, µs, ordre de grandeur). Après le dessin en point, Render n'est plus que le troisième poste : Present et Clear dominent.

`Render` n'est plus le poste dominant : dans le budget du moteur, `Clear` pèse plus du double, et `Present` (SDL, hors budget moteur) plus que tout le reste réuni. Conséquence pour la suite : la vectorisation (chantier B, A21) ne fera presque rien bouger **sur cette scène**, qui ne soumet plus que 1 388 faces par frame (≈ 20 µs selon le modèle). Elle sera mesurée sur sa **propre** scène (campagne V), où les faces dominent.

8. Les règles de mesure appliquées

Règle	Application ici
M1	Release uniquement pour les temps
M2	en-tête de CSV complet (scène, config, cpu, affinity, vues, résolution, overhead, `lodstats`)
M3	contrôle par T et par `Clear` entre références
M4	une variable à la fois : la campagne R en est l'application directe
M5	médiane / p95 / max, jamais la moyenne
M9	i7 hybride : P-core épinglé (`start /affinity 4`)
M19	`overhead_ns` comme thermomètre de fréquence
nouveau	un run de **comptage** (`STATS=1`) ne donne jamais de temps ; il dit **qui** paie, le run de temps dit **combien**

9. Ouvert

Point	Où	Quand
Séparer, dans les ≈ 204 µs restants, le coût des points de celui des vrais pixels	run `LV3_RASTER_STATS=1`	à l'occasion
Vectorisation de l'étage sommets et du test des faces	chantier B (A21), scènes V1 / V2	maintenant
Test des faces de dos en espace objet (idée scratchapixel)	chantier B	mesuré contre la transformation en lot
`Present` : verrouillage de texture, copie, synchronisation	EXE / SDL	chantier futur